Randomized average-case easiness of NP implies a one-sided subexponential algorithm for UP

Halley Goldberg, Valentine Kabanets, Zhenjian Lu, Igor C. Oliveira

doi:10.4230/LIPIcs.CCC.2022.16; CC BY 4.0

Editorial scope. Counted original Theorem37 (Theorem1 Item1) once. Randomized average-case easiness of NP is an essential retained assumption, and the final simulation has one-sided error. All non-routine author support needed for the pKt/UP proof is included, with imported DPG reconstruction stated exactly and not recursively reproved. Other rKt branches and other main applications are not claimed here.

Allowed standard prerequisites (editorial).

- Standard UP/NP and RAM program encodings, direct-product-generator reconstruction [11, Lemma3.14], elementary averaging and Markov bounds. Exact imported DPG Lemma14 retained. All source-local randomized heuristic, amplification, pKt reconstruction, conditional symmetry, unique-witness oracle elimination and depth-telescoping proofs retained. Unused rKt branches are not part of this pKt/UP proof.

2 Preliminaries (necessary author context)

2.1 Notation

We say that a function $p : \mathbb{N} \to \mathbb{N}$ is non-decreasing if $p(\ell) \geq \ell$ for every integer $\ell \geq 0$. Throughout the paper, we will rely on constructions whose associated complexity parameters (e.g., running time) can be bounded by some polynomial p. We will implicitly assume that the polynomials provided in these results are non decreasing in order to avoid including this condition in every statement. This can be done without loss of generality by the monotonicity of our bounds. For a probability distribution D_n, we use $\mathrm{supp}(D_n)$ to denote its support. Given an element x, we let $D_n(x)$ denote the probability of x under D_n. We denote by U_n the uniform distribution over n-bit strings. Let D_1 and D_2 be probability distributions supported over a set X, $A : X \to \{0,1\}$, and $\varepsilon \geq 0$. We say that A ε-distinguishes D_1 and D_2 if

$$\left| \Pr_{x \sim D_1}[A(x) = 1] - \Pr_{x \sim D_2}[A(x) = 1] \right| \geq \varepsilon.$$

If A is a randomized algorithm, each probability in the expression above also takes into account the internal randomness of A, which will be denoted by r_A.

2.2 Average-case complexity

Recall that a pair $(L, \mathcal{D})$ is a distributional problem if $L \subseteq \{0,1\}^*$ and $\mathcal{D} = \{D_n\}_{n\geq 1}$ is an ensemble of probability distributions, where each D_n is supported over $\{0,1\}^*$. Let $\mathcal{D} = \{D_n\}_{n\geq 1}$ be an ensemble of distributions. We say that $\mathcal{D} \in$ PSamp if there is a randomized polytime algorithm A such that, for every $n \geq 1$, $A(1^n)$ is distributed according to D_n. More generally, we use $D_n \in$ Samp$[T(n)]/a(n)$ to denote that D_n can be sampled by an algorithm that runs in time $T(n)$ and has advice complexity $a(n)$. We let DistNP denote the set of distributional problems $(L, \mathcal{D})$ with $L \in$ NP and $\mathcal{D} \in$ PSamp.

Definition 4 (Avg$_\delta$BPP **[3]).** Let $(L, \mathcal{D})$ be a distributional problem, and $\delta : \mathbb{N} \to [0,1]$. We say that $(L, \mathcal{D}) \in$ Avg$_\delta$BPP if there is randomized polytime algorithm A such that

1. for every $n > 0$, and every $x \in \text{supp}(D_n)$,
$$\Pr_{r_A}[A(x;n) \notin \{L(x), \perp\}] \leq 1/4,$$
2. for every $n > 0$,
$$\Pr_{x\sim D_n}\left[\Pr_{r_A}[A(x;n) = \perp] \geq 1/4\right] \leq \delta(n),$$

where r_A denotes the internal randomness of A. Such an algorithm A is called an randomized errorless heuristic for $(L, \mathcal{D})$ with failure probability at most δ.

Definition 5 (AvgBPP [3]). [1] We say that $(L, \mathcal{D})$ is in AvgBPP if there exist a randomized algorithm A and a polynomial p such that

1. for every $n, \delta > 0$, and every $x \in \text{supp}(D_n)$,
$$\Pr_{r_A}[A(x;n,\delta) \notin \{L(x), \perp\}] \leq 1/4,$$
2. for every $n, \delta > 0$,
$$\Pr_{x\sim D_n}\left[\Pr_{r_A}[A(x;n,\delta) = \perp] \geq 1/4\right] \leq \delta(n),$$
3. for every $n, \delta > 0$, and every $x \in \text{supp}(D_n)$, $A(x;n,\delta)$ runs in time at most $p(n/\delta)$.

Such an algorithm A is called a randomized errorless heuristic scheme for $(L, \mathcal{D})$.

Lemma 6. Let $(L, \mathcal{D})$ be a distributional problem in Avg$_{n^{-1}}$BPP. There exists a randomized polynomial-time algorithm B such that

1. for every $n > 0$, and every $x \in \text{supp}(D_n)$, $B(x;n) \in \{0,1\}$,
2. for every $n > 0$, and every $x \in \text{supp}(D_n)$, if $x \in L$, then $\Pr_{r_B}[B(x;n) = 1] \geq 1 - 1/n$, and
3. for every $n > 0$, $\Pr_{x\sim D_n, r_B}[B(x;n) = L(x)] \geq 1 - 3/n$.

[1] Some authors define AvgBPP as $\bigcap_{c>0}$ Avg$_{n^{-c}}$BPP. All our results also hold with respect to that definition.

Proof. Let A be an $\mathrm{Avg}_{n^{-1}}\mathrm{BPP}$ algorithm for $(L, \mathcal{D})$ as in Definition 4. Let A' be the algorithm obtained, using standard amplification techniques,[2] such that for all $x \in \mathrm{supp}(D_n)$,

$$\Pr_{r_{A'}}[A'(x;n) \notin \{L(x), \perp\}] \leq 1/n \tag{1}$$

and

$$\Pr_{x \sim D_n}\left[\Pr_{r_{A'}}[A'(x;n) = \perp] \geq 1/n\right] \leq 1/n. \tag{2}$$

Let B be the randomized algorithm which, on input $x \in \mathrm{supp}(D_n)$, simulates $A'(x;n)$ and outputs 1 if $A'(x;n) \in \{1, \perp\}$ or 0 if $A'(x;n) = 0$. By Equation (1), if $x \in L$, then

$$\Pr_{r_B}[B(x;n) = 0] = \Pr_{r_{A'}}[A'(x;n) \notin \{L(x), \perp\}] \leq 1/n.$$

Let $\mathrm{bad} := \{x \in \mathrm{supp}(D_n) \mid \Pr_{r_{A'}}[A'(x;n) = \perp] \geq 1/n\}$. By Equation (2), at most a $(1/n)$-measure of $x \in \mathrm{supp}(D_n)$ are in bad. Thus,

$$\begin{aligned}
\Pr_{x \sim D_n, r_B}[B(x;n) \neq L(x)] &\leq \Pr_{x, r_{A'}}[A'(x;n) = \perp] + \Pr_{x, r_{A'}}[A'(x;n) \notin \{L(x), \perp\}] \\
&\leq \Pr_{x}[x \in \mathrm{bad}] + \Pr_{x, r_{A'}}[A'(x;n) = \perp \mid x \notin \mathrm{bad}] \\
&\quad + \Pr_{x, r_{A'}}[A'(x;n) \notin \{L(x), \perp\}] \\
&\leq 1/n + 1/n + 1/n = 3/n.
\end{aligned}$$

This completes the proof. ◁

2.4 Direct product generators (DPGs)

Definition 13 (Direct Product Generator (DPG) [11, Definiton 3.10]). For $k, n \in \mathbb{N}$, we define the k-wise direct product generator to be the function

$$\mathrm{DP}_k : \{0,1\}^n \times \{0,1\}^{nk} \to \{0,1\}^{nk+k}$$

such that

$$\mathrm{DP}_k(x; z^1, \ldots, z^n) := (z^1, \ldots, z^k, x \cdot z^1, \ldots, x \cdot z^k).$$

Lemma 14 (DPG Reconstruction [11, Lemma 3.14]). For any $n, k \in \mathbb{N}$ with $k \leq 2n$, and $\varepsilon > 0$, there exists a pair of algorithms A and $\mathrm{Recon}^{(-)}$ such that

- Recon^{D} takes oracle access to an oracle $D : \{0,1\}^{nk+k} \to \{0,1\}$.
- $A : \{0,1\}^n \times \{0,1\}^r \to \{0,1\}^k$ is called the advice function and is computable in time $\mathrm{poly}(n/\varepsilon)$.
- $\mathrm{Recon}^{(-)} : \{0,1\}^k \times \{0,1\}^r \to \{0,1\}^n$ is called a reconstruction procedure and is computable in time $\mathrm{poly}(n/\varepsilon)$.[3]

[2] In more detail, A' can be obtained from A by running it $O(n)$ times and selecting the most common output.

[3] We note that if the oracle is a uniform algorithm that runs in time $t(m)$ on inputs of length m, the reconstruction procedure can be computed in time $t(nk+k) \cdot \mathrm{poly}(n/\varepsilon)$.

- The randomness complexity r is at most $\text{poly}(n/\varepsilon)$.
- For any string $x \in \{0,1\}^n$ and any function D that ε-distinguishes $\text{DP}_k(x; U_{nk})$ from U_{nk+k}, it holds that

$$\Pr_{w \sim \{0,1\}^r}[\text{Recon}^D(A(x,w),w) = x] \geq 1/\text{poly}(n/\varepsilon).$$

2.7 Encoding tuples (author context)

We will also need to consider distributions that generate input instances that are tuples, while the only known information to the distributions is the length of the instances. To do this, we will encode the format as the length of the instances. More specifically, we use the following way to encode tuples [3].

Proposition 16 ([3]; see also [6, Proposition 3.1]). There is a pair of polytime encoding-decoding algorithms $\text{Enc} : \mathbb{N}^* \to \mathbb{N}$ and $\text{Dec} : \mathbb{N} \to \mathbb{N}^* \cup \{\perp\}$ such that

1. For $k \in \mathbb{N}$ and k integers $n_1, n_2, \ldots, n_k$, we write $\langle n_1, n_2, \ldots, n_k \rangle := \text{Enc}(n_1, n_2, \ldots, n_k)$, and we have
   (a) $\sum_{i=1}^{k}(2 \cdot (\lfloor \log n_i \rfloor + 2) + n_i) \leq \langle n_1, \ldots, n_k \rangle \leq O_k(\prod_{i=1}^{k}(n_i \cdot \log^2 n_i))$.
   (b) $\text{Dec}(\langle n_1, n_2, \ldots, n_k \rangle) = n_1, \ldots, n_k$. Here, Dec takes the binary representation of an integer and outputs k integers which are also represented in binary.
2. $\text{Dec}(u) = \perp$ if $u \neq \langle \vec{n} \rangle$ for any $\vec{n} \in \mathbb{N}^*$.

Proof. Given k integers $n_1, n_2, \ldots, n_k$, the encoding algorithm Enc first obtains the binary string

$$z := \alpha_1 \circ 01 \circ \alpha_2 \circ 01 \circ \alpha_k \circ 01 \circ \text{bin}(n_1) \circ \text{bin}(n_2) \circ \cdots \circ \text{bin}(n_k),$$

where for each $i \in [k]$, α_i is obtained from $\text{bin}(|\,\text{bin}(n_i)|)$ with every bit doubled. Then Enc outputs the integer n whose binary representation is z. It is easy to see that given the binary representation of n, which is simply the string z, one can recover the $n_1, n_2, \ldots, n_k$ in time polynomial in $|z|$. Also, item (a) can be verified by a simple calculation. ◁

3 Probabilistic Time-Bounded Kolmogorov Complexity (author excerpts)

In this section, we formally define probabilistic Kolmogorov complexity and prove some useful properties of this notion, which will be used later in proving our main results.

Definition 17 (pK^t). For strings $x, y \in \{0,1\}^*$, a time bound $t \in \mathbb{N}$, an oracle A, and $\delta \in [0,1]$, the δ-probabilistic A-oracle t-bounded Kolmogorov complexity of x given y is defined as

$$\mathrm{pK}^{t,A}_\delta(x \mid y) := \min\left\{k \;\middle|\; \Pr_{w\sim\{0,1\}^t}[\exists M \in \{0,1\}^k, M^A(w,y) \text{ outputs } x \text{ within } t \text{ steps}] \geq \delta\right\},$$

where M is a RAM-machine.[4] We omit the subscript δ if $\delta = 2/3$, omit the superscript A if $A = \emptyset$, omit "$\mid y$" if y is the empty string, and omit the superscript t if $t = \infty$. For simplicity, in the rest of this section we only consider pK^t without oracles. It is easy to see that all the results also hold with any oracle.

Lemma 21 (Success Amplification). For any string $x \in \{0,1\}^n$, time bound $t \in \mathbb{N}$, and $0 \leq \alpha < \beta \leq 1$, we have

$$\mathrm{pK}^{O(qt/\alpha)}_\beta(x) \leq \mathrm{pK}^t_\alpha + O(\log(q/\alpha)),$$

where $q = \ln(1/(1-\beta))$.

Proof. Suppose $\mathrm{pK}^t_\alpha(x) = k$. Then by definition, we have

$$\Pr_{w\sim\{0,1\}^t}[\exists M_w \in \{0,1\}^k \text{ such that } M_w(w) \text{ outputs } x \text{ within } t \text{ steps}] \geq \alpha.$$

Call such a w good. After sampling ℓ independent $w^{(1)}, w^{(2)}, \ldots, w^{(\ell)} \in \{0,1\}^t$, the probability that no $w^{(i)}$ is good is at most $(1-\alpha)^\ell \leq e^{-\alpha\ell}$, which can be made at most $1-\beta$ by choosing $\ell = q/\alpha$. It follows that, with probability at least β over a random $w = (w^{(1)}, w^{(2)}, \ldots, w^{(q/\alpha)}) \in \{0,1\}^{qt/\alpha}$, there exists an index $1 \leq i \leq q/\alpha$ (described with at most $\lceil \log(q/\alpha) \rceil$ bits) and a program $M_{w^{(i)}} \in \{0,1\}^k$ such that $M_{w^{(i)}}(w^{(i)})$ outputs x within $O(qt/\alpha)$ steps. Hence, $\mathrm{pK}^{O(qt/\alpha)}_\beta(x) \leq k + O(\log(q/\alpha))$. ◁ Note that the parameter α affects the time complexity in the resulting $\mathrm{pK}^{O(qt/\alpha)}_\beta$ bound. Lemma 21 is sufficient in applications where $\alpha \geq 1/\operatorname{poly}(n)$.

3.2 Bounding pK^t via DPG reconstruction (selected original case)

The following is a key lemma for pK^t that we will use in proving our main results related to randomized average-case complexity.

Lemma 22 (pK^t Reconstruction Lemma). For $\varepsilon > 0$, $x \in \{0,1\}^n$, $s \in \mathbb{N}$, and $k \in \mathbb{N}$ satisfying $k \leq 2n$, let D be a randomized algorithm that takes an advice string β and runs in time t_D such that D ε-distinguishes $\mathrm{DP}_k(x; U_{nk})$ from U_{nk+k}. Then there is a polynomial p_{DP} such that

$$\mathrm{pK}^{\widetilde{O}(t_D)\cdot p_{\mathrm{DP}}(n/\varepsilon)}(x \mid \beta) \leq k + \log p_{\mathrm{DP}}(nt_D/\varepsilon).$$

[4] We use the RAM model in this definition for convenience: it simplifies the time bound estimates for a machine M that takes as input both randomness w and an "auxiliary" string y, and allows both w and y to be as long as the running time of the RAM-machine M; cf. [29] for a similar definition in the case of deterministic time-bounded conditional Kolmogorov complexity. Alternatively, we could use a TM model, with inputs w and y presented on two different tapes. However, we prefer the RAM model as it's also useful in the context of fine-grained worst-case to average-case reductions [6].

Proof. We will view the distinguisher D as a deterministic algorithm that takes a string of t_D bits, denoted by r_D, as its internal randomness. By our assumption (and dropping the absolute value sign without loss of generality), we have

$$\mathbb{E}_{y\sim\{0,1\}^{nk},r_D\sim\{0,1\}^{t_D}}[D(\mathrm{DP}_k(x,y);r_D)]-\mathbb{E}_{z\sim\{0,1\}^{nk+k},r_D\sim\{0,1\}^{t_D}}[D(z;r_D)]\geq\varepsilon.$$

Then by an averaging argument, we have

$$\Pr_{r_D}[\mathbb{E}_y[D(\mathrm{DP}_k(x,y);r_D)]-\mathbb{E}_z[D(z;r_D)]\geq\varepsilon/2]\geq\varepsilon/2. \tag{4}$$

In other words, with probability at least $\varepsilon/2$ over the internal randomness r_D, $D(-,r_D)$ is an $(\varepsilon/2)$-distinguisher for $\mathrm{DP}_k(x;U_{nk})$ and U_{nk+k}. Let us say that r_D is good if this is true. Let $\mathrm{Recon}^{(-)}:\{0,1\}^k\times\{0,1\}^r\to\{0,1\}^n$ be the reconstruction procedure in Lemma 14 that works for distinguishing parameter $\varepsilon/2$, where the randomness complexity r is $\mathrm{poly}(n/\varepsilon)$. We have

$$\begin{aligned}
&\Pr_{w\sim\{0,1\}^r,r_D\sim\{0,1\}^{t_D}}[\exists\alpha\in\{0,1\}^k \text{ such that } \mathrm{Recon}^{D(-,r_D)}(\alpha,w)=x]\\
&\geq \Pr_{w,r_D}[\exists\alpha\in\{0,1\}^k \text{ such that } \mathrm{Recon}^{D(-,r_D)}(\alpha,w)=x \mid r_D \text{ is good}]\cdot\Pr_{r_D}[r_D \text{ is good}]\\
&\geq \frac{1}{\mathrm{poly}(n/\varepsilon)}\cdot(\varepsilon/2). \qquad \text{(Lemma 14 and Equation (4))}
\end{aligned}$$

The above implies that for at least $1/\mathrm{poly}(n/\varepsilon)$ fraction of the randomness (w,r_D), there exists a program M, which takes k bits for some α (that can depend on the randomness) and the advice of β used by D, such that M simulates $\mathrm{Recon}^{D(-,r_D)}$ on (α,w) and outputs x. Note that since Recon^D runs in time $t_D\cdot\mathrm{poly}(n/\varepsilon)$, our program M can be made to run in time $T:=\widetilde{O}(t_D)\cdot\mathrm{poly}(n/\varepsilon)$. Therefore, we have

$$\mathrm{pK}^T_{1/\mathrm{poly}(n/\varepsilon)}(x\mid\beta)\leq k+O(\log(nt_D/\varepsilon)).$$

Then the lemma follows easily from Lemma 21. ◁

3.4 Symmetry of information (original Item1 and its full proof)

Lemma 26 (Symmetry of information for pK^t and rK^t under the average-case easiness of NP), Item 1. If DistNP $\subseteq$ AvgBPP, then there exist polynomials p and p_0 such that for all sufficiently large $n,m\in\mathbb{N}$, all $t\geq p_0(n,m)$, and all $0\leq\tau\leq t$ the following holds: for every $x\in\{0,1\}^n$ and every family $\{y_r\in\{0,1\}^m\mid r\in\{0,1\}^\tau\}$, with probability at least $9/10$ over $r\in\{0,1\}^\tau$,

$$\mathrm{pK}^t(x,y_r\mid r)>\mathrm{pK}^{p(t)}(x\mid r)+\mathrm{pK}^{p(t)}(y_r\mid x,r)-\log p(t).$$

Moreover, the special case without conditioning on r also holds for both items above, i.e., when r is the empty string (for $\tau=0$) and y is an arbitrary single string.

Proof of Item 1. Define a language

$$L := \{(u, v, w, w', 1^s) \mid \exists M \in \{0,1\}^s, M(w, w') \text{ prints } uv \text{ in } |w| \text{ steps, and } s = |u|+|v|-10\}.$$

Note that $L \in$ NP. Define a distribution family $\mathcal{D} = \{D_{\langle nk+k, mk'+k', 2t, \tau, s\rangle}\}$ as follows: sample $u \sim U_{nk+k}$, $v \sim U_{mk'+k'}$, $w \sim U_{2t}$, and $w' \sim U_\tau$, and then output $(u, v, w, w', 1^s)$. Note that the ensemble $\mathcal{D}$ is in PSamp under an appropriate encoding of its defining tuple (for simplicity, we omit the padding in the output of $\mathcal{D}$; see Section 2.1). Using the assumption that DistNP $\subseteq$ AvgBPP, it follows that $(L, \mathcal{D}) \in$ AvgBPP. Let B be a randomized algorithm for $(L, \mathcal{D})$ as in Lemma 6. Let $x \in \{0,1\}^n$ and $\{y_r\}_r \subseteq \{0,1\}^m$ be given (for sufficiently large n and m). Let $k, k' \in \mathbb{N}$ be arbitrary parameters such that $k \leq n$ and $k' \leq m$. Observe that there exists a polynomial p_0 such that for any $t \geq p_0(n, m)$, some constant d, and all $z \in \{0,1\}^{nk}$, $z' \in \{0,1\}^{mk'}$, and $r \in \{0,1\}^\tau$,

$$\mathrm{pK}^{2t}(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z') \mid r) \leq \mathrm{pK}^t(x, y_r \mid r) + |z| + |z'| + d \log t. \tag{6}$$

Here $p_0(n, m)$ reflects the time required to deterministically compute $(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'))$ given x, y_r, z, z', and $d \log t$ bits of information to delineate x from y_r. Let $t \geq p_0(n, m)$. By a counting argument, for independently random u, v, w, and w',

$$\Pr_{u,v,w,w'}[(u, v, w, w', 1^s) \in L] \leq \frac{2^s \cdot 2^{|w|+|w'|}}{2^{|u|+|v|+|w|+|w'|}} = 2^{s-|u|-|v|} = 2^{-10},$$

where the last equality is by the definition of L requiring that $s = |u| + |v| - 10$. Then

$$\begin{aligned}\Pr_{u,v,w,w',r_B}[B(u, v, w, w', 1^s) = 1] \leq &\Pr_{u,v,w,w'}[(u, v, w, w', 1^s) \in L] \\ &+ \Pr_{u,v,w,w',r_B}[B(u, v, w, w', 1^s) \neq L(u, v, w, w', 1^s)] \\ &\leq 2^{-10} + (3/n). \end{aligned} \tag{7}$$

By Markov's inequality, for at least 9/10 fraction of random strings $r \in \{0,1\}^\tau$, we get

$$\Pr_{u,v,w,r_B}[B(u, v, w, r, 1^s) = 1] \leq 10 \cdot (2^{-10} + (3/n)) \leq 1/10. \tag{8}$$

Fix any such r so that Equation (8) holds. This also fixes the string y_r. Next we show, using a hybrid argument, that $B(-, U_{2t}, r, 1^s)$ cannot distinguish between the uniform distribution and the distribution $(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'))$, for random independent z, z', where $k \approx \mathrm{pK}^{p_{\mathrm{DP}}(t)}(x \mid r)$ and $k' \approx \mathrm{pK}^{p_{\mathrm{DP}}(t)}(y_r \mid x, r)$. This will imply that

$$\Pr_w[(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'), w, r, 1^s) \in L] < 2/3$$

for some z, z', yielding the desired lower bound on $\mathrm{pK}^{2t}(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z') \mid r)$ via our choice of s and the definition of L. We give the details of the hybrid argument next. First, toward a contradiction, suppose

$$\Pr_{z,v,w,r_B}[B(\mathrm{DP}_k(x; z), v, w, r, 1^s) = 1] > 1/2.$$

In this case, comparing with Equation (8), we get a randomized distinguisher (with advice) for $\mathrm{DP}_k(x; U_{nk})$, defined by sampling $v \sim U_{mk'+k'}$ and $w \sim U_{2t}$, and outputting $B(-, v, w, r, 1^s)$. By Lemma 22,

$$\mathrm{pK}^{q(t)}(x \mid r) \leq k + \log q(t) \tag{9}$$

for some polynomial q such that $q(t) \geq p_{\mathrm{DP}}(t_B \cdot 3 \cdot n)$ whenever $t \geq n + m$ and $s \leq n^3$, where p_{DP} is the polynomial from Lemma 22 and t_B denotes the time required to compute $B(-, v, w, r, 1^s)$. Define $k := \mathrm{pK}^{q(t)}(x \mid r) - \log q(t) - 1$ so that Equation (9) does not hold. Assume for now that $k > 0$. Hence,

$$\Pr_{z,v,w,r_B}[B(\mathrm{DP}_k(x; z), v, w, r, 1^s) = 1] \leq 1/2. \tag{10}$$

Again, toward a contradiction, suppose that for all z, z',

$$\Pr_w[(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'), w, r, 1^s) \in L] \geq 2/3.$$

By definition of B, this implies that

$$\Pr_{z,z',w,r_B}[B(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'), w, r, 1^s) = 1] \geq (2/3)(1 - 1/n) > 5/8.$$

In this case, comparing with Equation (10), we get a randomized distinguisher (with advice) B' for $\mathrm{DP}_{k'}(y_r; U_{mk'})$, defined by sampling $z \sim U_{nk}$, $w \sim U_{2t}$, and outputting $B(\mathrm{DP}_k(x; z), -, w, r, 1^s)$. By Lemma 22,

$$\mathrm{pK}^{q'(t)}(y_r \mid x, r) \leq k' + \log q'(t) \tag{11}$$

for some polynomial q' with $q'(t) \geq p_{\mathrm{DP}}(t_{B'} \cdot 8 \cdot m)$ whenever $t \geq n + m$ and $s \leq n^3$, where $t_{B'}$ denotes the time required to compute B'. We now choose $k' := \mathrm{pK}^{q'(t)}(y_r \mid x, r) - \log q'(t) - 1$ so that Equation (11) does not hold. Assume for now that $k' > 0$. Hence, there exist z and z' such that

$$\begin{aligned}&\Pr_w[(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z'), w, r, 1^s) \in L] \\ &= \Pr_w[\exists M \in \{0,1\}^s, M(w, r) \text{ outputs } (\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z')) \text{ within } |w| \text{ steps}] < 2/3,\end{aligned}$$

which implies that

$$\mathrm{pK}^{2t}(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z') \mid r) > s.$$

For this choice of z, z', by definition of s we get that $s = |u| + |v| - 10 = |z| + k + |z'| + k' - 10$, and so

$$\mathrm{pK}^{2t}(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z') \mid r) > |z| + k + |z'| + k' - 10.$$

Combining this inequality with Equation (6), we get

$$\begin{aligned}\mathrm{pK}^t(x, y_r \mid r) &\geq \mathrm{pK}^{2t}(\mathrm{DP}_k(x; z), \mathrm{DP}_{k'}(y_r; z') \mid r) - |z| - |z'| - d \log t \\ &> k + k' - d \log t - 10.\end{aligned} \tag{12}$$

The definitions of k and k' then imply that

$$\begin{aligned}\mathrm{pK}^t(x, y_r \mid r) > \mathrm{pK}^{q(t)}(x \mid r) + \mathrm{pK}^{q'(t)}(y_r \mid x, r) \\ - \log q(t) - \log q'(t) - d \log t - 12.\end{aligned}$$

For the polynomial $p(t) := (t^{d+1}) \cdot q(t) \cdot q'(t)$ and for every $t \geq p_0(|x|, |y|)$, we get

$$\mathrm{pK}^t(x, y_r \mid r) > \mathrm{pK}^{p(t)}(x \mid r) + \mathrm{pK}^{p(t)}(y_r \mid x, r) - \log p(t),$$

as desired. Finally, consider the case that $k \leq 0$ or $k' \leq 0$. If $k \leq 0$, then $\mathrm{pK}^{q(t)}(x \mid r) \leq \log q(t) + 1$, implying that $\mathrm{pK}^{p(t)}(x \mid r) < \log p(t)$. But then the lemma simply follows from the fact that $\mathrm{pK}^t(x, y_r \mid r) \geq \mathrm{pK}^{p(t)}(y_r \mid x, r)$. Similarly, if $k' \leq 0$, then $\mathrm{pK}^{p(t)}(y_r \mid x, r) < \log p(t)$, and the lemma follows from the fact that $\mathrm{pK}^t(x, y_r \mid r) \geq \mathrm{pK}^{p(t)}(x \mid r)$. ◁

3.7 Probabilistic computational depth (original Item1 and its full proof)

The next lemma is a straightforward adaptation of an argument from [11]. We provide a proof for completeness.

Lemma 31 (Computational Depth Upper Bound), Item 1. For all $x \in \{0,1\}^n$ and $r \in \{0,1\}^*$, and for all non-decreasing polynomials p_0 and p, if n is large enough then there exists a time bound t_1 such that $p_0(n) \leq t_1 \leq 2^{n/\log n}$ and

$$\mathrm{pK}^{t_1}(x \mid r) - \mathrm{pK}^{p(t_1)}(x \mid r) \leq O(n/\log n).$$

Proof. We will start by proving Item 1. Given $x \in \{0,1\}^n$ and polynomials p_0 and p, define the polynomial $\tau := p \circ p_0$. For an integer $I \geq 1$, consider the following telescoping sum:

$$\begin{aligned}\mathrm{pK}^{\tau(n)}(x \mid r) - \mathrm{pK}^{\tau^{I+1}(n)}(x \mid r) = {} & (\mathrm{pK}^{\tau(n)}(x \mid r) - \mathrm{pK}^{\tau^2(n)}(x \mid r)) \\ & + (\mathrm{pK}^{\tau^2(n)}(x \mid r) - \mathrm{pK}^{\tau^3(n)}(x \mid r)) \\ & + \cdots + (\mathrm{pK}^{\tau^I(n)}(x \mid r) - \mathrm{pK}^{\tau^{I+1}(n)}(x \mid r)),\end{aligned}$$

where $\tau^i(-)$ denotes the composition of τ with itself i times. For any choice of x, p_0, and p as in the statement of the lemma, $\mathrm{pK}^{\tau(n)}(x \mid r) \leq n + d$, for some universal constant $d \geq 0$; hence, the above sum is at most $n + d$. By averaging, there is some index $i_0 \in [I]$ such that

$$\mathrm{pK}^{\tau^{i_0}(n)}(x \mid r) - \mathrm{pK}^{\tau^{i_0+1}(n)}(x \mid r) \leq (n+d)/I.$$

For this i_0, define $t_1 := \tau^{i_0}(n)$. Note that $t_1 \geq \tau(n) \geq p_0(n)$, since $i_0 \geq 1$ and $p(\ell) \geq \ell$ for every input ℓ. Letting $c \in \mathbb{N}$ be such that $\tau(n) \leq n^c$ for sufficiently large n, define $I := \log_c(n/(\log_2 n)^2)$. Then $t_1 \leq n^{c^I} = 2^{n/\log n}$. Moreover,

$$\begin{aligned}\mathrm{pK}^{t_1}(x \mid r) - \mathrm{pK}^{p(t_1)}(x \mid r) &\leq \mathrm{pK}^{t_1}(x \mid r) - \mathrm{pK}^{\tau(t_1)}(x \mid r) \\ &\leq O(n/\log n),\end{aligned}$$

as desired.

4 Probabilistic Worst-Case to Average-Case Reductions (UP case)

We will also need the following analogue of Lemma 33 for the case of unique witnesses.

Lemma 35 (Oracle Elimination for UP). Suppose DistNP $\subseteq$ AvgBPP, and let $L \in$ UP. Let $x \in L_n$ be sufficiently large, and let $y_x \in \{0,1\}^{n^c}$ be the unique L-witness for x, for some constant $c > 0$. There exists a polynomial q such that for all $t \geq n + n^c$,

$$\mathrm{pK}^{q(t)}(x, y_x) \leq \mathrm{pK}^t(x) + \log q(t).$$

Proof. Let $L \in$ UP with deterministic verifier V running in time n^c on inputs $x \in \{0,1\}^n$ and $y \in \{0,1\}^{n^c}$. Let

$$L' := \{(\mathrm{DP}_k(x, y; z), w, 1^s, 1^n) \mid \exists M \in \{0,1\}^s,\ M(w) \text{ outputs } x \in \{0,1\}^n \text{ within } |w| \text{ steps}, \\ \text{where } k = s + 10, \text{ and } V(x, y) = 1\}.$$

Note that $L' \in$ NP. Define a distribution family $\mathcal{D} = \{D_{\langle (n+n^c)k+k, t, s, n\rangle}\}$ as follows: sample $u \sim U_{(n+n^c)k+k}$, $w \sim U_t$, and output $(u, w, 1^s, 1^n)$. By assumption, $(L', \mathcal{D}) \in$ AvgBPP. Let B be a randomized heuristic algorithm for $(L', \mathcal{D})$ as described in Lemma 6. Let $x \in L_n$ be sufficiently large, and let $y_x \in \{0,1\}^{n^c}$ be the unique L-witness for x. Define $s := \mathrm{pK}^t(x)$, and let $t \geq n + n^c$. By definition of pK^t and L', for any $z \in \{0,1\}^{(n+n^c)k}$,

$$\Pr_w[(\mathrm{DP}_k(x, y_x; z), w, 1^s, 1^n) \in L'] \geq 2/3.$$

Then by definition of B,

$$\begin{aligned}\Pr_{z,w,r_B}[B(\mathrm{DP}_k(x, y_x; z), w, 1^s, 1^n) = 1] &\geq \Pr_{z,w}[(\mathrm{DP}_k(x, y_x; z), w, 1^s, 1^n) \in L'] \\ &\quad \cdot \Pr_{r_B}[B(\omega) = L'(\omega) \mid \omega \in L'] \\ &\geq \frac{2}{3}(1 - (1/n)) \geq 1/2. \qquad (22)\end{aligned}$$

On the other hand, for u and w selected uniformly at random,

$$\begin{aligned}\Pr_{u,w}[(u, w, 1^s, 1^n) \in L'] &= \Pr_{u,w}[\exists (x, y) \in \{0,1\}^{n+n^c}, \exists z \in \{0,1\}^{(n+n^c)k}, \exists M \in \{0,1\}^s, \\ &\qquad M(w) = x \wedge V(x, y) = 1 \wedge u = \mathrm{DP}_k(x, y; z)] \\ &\leq \frac{2^{s+|w|+|z|}}{2^{k+|w|+|z|}} = 2^{-10},\end{aligned}$$

where the inequality is by a union bound and a counting argument, and the last equality by definition of $k = s + 10 = \mathrm{pK}^t(x) + 10$. Then

$$\begin{aligned}\Pr_{u,w,r_B}[B(u, w, 1^s, 1^n) = 1] &\leq \Pr_{u,w}[(u, w, 1^s, 1^n) \in L'] \\ &\quad + \Pr_{u,w,r_B}[B(u, w, 1^s, 1^n) \neq L'(u, w, 1^s, 1^n)] \\ &\leq 2^{-10} + (3/n) \leq 1/10.\end{aligned}$$

Comparing with Equation (22), it is clear that $B(-, U_t, 1^s, 1^n)$ is a randomized distinguisher for $\mathrm{DP}_k(x, y_x; U_{(n+n^c)k})$. Lemma 22 implies that

$$\begin{aligned}\mathrm{pK}^{p'(t)}(x, y_x) &\leq k + \log p'(t) \\ &= \mathrm{pK}^t(x) + 10 + \log p'(t)\end{aligned}$$

for some polynomial p' with $p'(t) \geq p_{\mathrm{DP}}(t_b \cdot 3 \cdot (n + n^c))$, where p_{DP} is the polynomial from Lemma 22 and t_B denotes the time required to compute $B(-, U_t, 1^s, 1^n)$. It follows that $\mathrm{pK}^{q(t)}(x, y_x) \leq \mathrm{pK}^t(x) + \log q(t)$ for the polynomial $q(t) := t \cdot p'(t)$. ◁

4.3 Case of UP

Theorem 37. If DistNP $\subseteq$ AvgBPP, then UP $\subseteq$ RTIME$[2^{O(n/\log n)}]$.

Proof. Let $L \in$ UP, $x \in L_n$ for n sufficiently large, and $y_x \in \{0,1\}^{n^c}$ the unique L-witness for x, for some constant $c > 0$. Let V be a deterministic verifier for L running in time n^c. Let p_0, p, and q be the polynomials from Lemmas 26 (Item 1) and 35 respectively. By Lemma 31, there exists $p_0(n+n^c) \leq t \leq 2^{n/\log n}$ such that

$$\mathrm{pK}^t(x) - \mathrm{pK}^{p(q(t))}(x) \leq O(n/\log n).$$

For such a t, combining Lemma 26 and Lemma 35 as in the proof of Lemma 34, we obtain

$$\mathrm{pK}^{p(q(t))}(y_x \mid x) \leq (\mathrm{pK}^t(x) - \mathrm{pK}^{p(q(t))}(x)) + 2\log p(q(t)).$$

The above inequalities imply that for some constant $d \in \mathbb{N}$,

$$\mathrm{pK}^{2^{dn/\log n}}(y_x \mid x) \leq dn/\log n.$$

We are now ready to define a probabilistic algorithm A deciding L.

- On input $x \in \{0,1\}^n$, A samples $w \sim \{0,1\}^{2^{dn/\log n}}$. It then exhaustively searches over all machines $M \in \{0,1\}^{dn/\log n}$, running $M(w, x)$ for $2^{dn/\log n}$ steps to produce some output y. A accepts iff a y is obtained such that $V(x, y) = 1$.

If $x \in L$, then A accepts with probability at least $2/3$ over w; if $x \notin L$, then A never accepts. ◁

References

[1] Luis Filipe Coelho Antunes and Lance Fortnow. Worst-case running times for average-case algorithms. In Proceedings of the 24th Annual IEEE Conference on Computational Complexity, CCC 2009, Paris, France, 15-18 July 2009, pages 298–303. IEEE Computer Society, 2009. doi:10.1109/CCC.2009.12.

[2] Shai Ben-David, Benny Chor, Oded Goldreich, and Michael Luby. On the theory of average case complexity. J. Comput. Syst. Sci., 44(2):193–219, 1992. doi:10.1016/0022-0000(92)90019-F.

[3] Andrej Bogdanov and Luca Trevisan. Average-case complexity. Found. Trends Theor. Comput. Sci., 2(1), 2006. doi:10.1561/0400000004.

[4] Andrej Bogdanov and Luca Trevisan. On worst-case to average-case reductions for NP problems. SIAM J. Comput., 36(4):1119–1159, 2006. doi:10.1137/S0097539705446974.

[5] Harry Buhrman, Lance Fortnow, and Aduri Pavan. Some results on derandomization. Theory Comput. Syst., 38(2):211–227, 2005. doi:10.1007/s00224-004-1194-y.

[6] Lijie Chen, Shuichi Hirahara, and Neekon Vafa. Average-case hardness of NP and PH from worst-case fine-grained assumptions. In Mark Braverman, editor, 13th Innovations in Theoretical Computer Science Conference, ITCS 2022, January 31 - February 3, 2022, Berkeley, CA, USA, volume 215 of LIPIcs, pages 45:1–45:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ITCS.2022.45.

[7] Halley Goldberg and Valentine Kabanets. A simpler proof of the worst-case to average-case reduction for polynomial hierarchy via symmetry of information. Electron. Colloquium Comput. Complex., 7:1–14, 2022. URL: https://eccc.weizmann.ac.il/report/2022/007/.

[8] Oded Goldreich. The Foundations of Cryptography - Volume 1: Basic Techniques. Cambridge University Press, 2001. doi:10.1017/CBO9780511546891.

[9] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing, STOC '89, pages 25–32, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/73007.73010.

[10] Shuichi Hirahara. Characterizing average-case complexity of PH by worst-case meta-complexity. In Sandy Irani, editor, 61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020, pages 50–60. IEEE, 2020. doi:10.1109/FOCS46700.2020.00014.

[11] Shuichi Hirahara. Average-case hardness of NP from exponential worst-case hardness assump- tions. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 292–302. ACM, 2021. doi:10.1145/3406325.3451065.

[12] Shuichi Hirahara. Symmetry of information in heuristica. Manuscript, 2021.

[13] Shuichi Hirahara and Mikito Nanashima. On worst-case learning in relativized heuristica. In Symposium on Foundations of Computer Science (FOCS), 2021.

[14] Shuichi Hirahara and Rahul Santhanam. Errorless versus error-prone average-case complexity. In Mark Braverman, editor, 13th Innovations in Theoretical Computer Science Conference, ITCS 2022, January 31 - February 3, 2022, Berkeley, CA, USA, volume 215 of LIPIcs, pages

84:1–84:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ITCS.2022.84.

[15] Shuichi Hirahara and Rahul Santhanam. Excluding PH pessiland. In Mark Braverman, editor, 13th Innovations in Theoretical Computer Science Conference, ITCS 2022, January 31 - February 3, 2022, Berkeley, CA, USA, volume 215 of LIPIcs, pages 85:1–85:25. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ITCS.2022.85.

[16] Russell Impagliazzo. A personal view of average-case complexity. In Proceedings of the Tenth Annual Structure in Complexity Theory Conference, Minneapolis, Minnesota, USA, June 19-22, 1995, pages 134–147, 1995. doi:10.1109/SCT.1995.514853.

[17] Russell Impagliazzo. Relativized separations of worst-case and average-case complexities for NP. In Proceedings of the 26th Annual IEEE Conference on Computational Complexity, CCC 2011, San Jose, California, USA, June 8-10, 2011, pages 104–114, 2011. doi:10.1109/CCC.2011.34.

[18] Russell Impagliazzo and Leonid A. Levin. No better ways to generate hard NP instances than picking uniformly at random. In 31st Annual Symposium on Foundations of Computer Science, St. Louis, Missouri, USA, October 22-24, 1990, Volume II, pages 812–821. IEEE Computer Society, 1990. doi:10.1109/FSCS.1990.89604.

[19] Russell Impagliazzo and Avi Wigderson. P = BPP if E requires exponential circuits: De- randomizing the XOR lemma. In Frank Thomson Leighton and Peter W. Shor, editors, Proceedings of the Twenty-Ninth Annual ACM Symposium on the Theory of Computing, El Paso, Texas, USA, May 4-6, 1997, pages 220–229. ACM, 1997. doi:10.1145/258533.258590.

[20] Michael J. Kearns, Robert E. Schapire, and Linda Sellie. Toward efficient agnostic learning. Mach. Learn., 17(2-3):115–141, 1994. doi:10.1007/BF00993468.

[21] Pravesh K. Kothari and Roi Livni. Improper learning by refuting. In Anna R. Karlin, editor, 9th Innovations in Theoretical Computer Science Conference, ITCS 2018, January 11-14, 2018, Cambridge, MA, USA, volume 94 of LIPIcs, pages 55:1–55:10. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. doi:10.4230/LIPIcs.ITCS.2018.55.

[22] Clemens Lautemann. BPP and the polynomial hierarchy. Inf. Process. Lett., 17(4):215–217, 1983. doi:10.1016/0020-0190(83)90044-3.

[23] Troy Lee. Kolmogorov complexity and formula lower bounds. PhD thesis, University of Amsterdam, 2006.

[24] Leonid A. Levin. Laws of information conservation (nongrowth) and aspects of the foundation of probability theory. Problemy Peredachi Informatsii, 10(3):30–35, 1974.

[25] Leonid A. Levin. Average case complete problems. SIAM J. Comput., 15(1):285–286, 1986. doi:10.1137/0215020.

[26] Ming Li and Paul M. B. Vitányi. An introduction to Kolmogorov complexity and its applications. Springer, 2019.

[27] Ming Li and Paul M. B. Vitányi. An Introduction to Kolmogorov Complexity and Its Applications, 4th Edition. Texts in Computer Science. Springer, 2019. doi:10.1007/978-3-030-11298-1.

[28] Yanyi Liu and Rafael Pass. On one-way functions and kolmogorov complexity. In 61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020, pages 1243–1254, 2020. doi:10.1109/FOCS46700.2020.00118.

[29] Yanyi Liu and Rafael Pass. On one-way functions from np-complete problems. Electron. Colloquium Comput. Complex., page 59, 2021. URL: https://eccc.weizmann.ac.il/report/2021/059.

[30] Yanyi Liu and Rafael Pass. On the possibility of basing cryptography on EXP $\neq$ BPP. In Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I, pages 11–40, 2021. doi:10.1007/978-3-030-84242-0_2.

[31] Luc Longpré and Osamu Watanabe. On symmetry of information and polynomial time invertibility. Inf. Comput., 121(1):14–22, 1995. doi:10.1006/inco.1995.1120.

[32] Zhenjian Lu and Igor C. Oliveira. An efficient coding theorem via probabilistic representations and its applications. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, 48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, July 12-16, 2021, Glasgow, Scotland (Virtual Conference), volume 198 of LIPIcs, pages 94:1–94:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ICALP.2021.94.

[33] Zhenjian Lu, Igor C. Oliveira, and Rahul Santhanam. Pseudodeterministic algorithms and the structure of probabilistic time. In Samir Khuller and Virginia Vassilevska Williams, editors, STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 303–316. ACM, 2021. doi:10.1145/3406325.3451085.

[34] Zhenjian Lu, Igor C. Oliveira, and Marius Zimand. Optimal coding theorems in time-bounded kolmogorov complexity. In 49th International Colloquium on Automata, Languages and Programming, 2022.

[35] Peter Bro Miltersen, N. V. Vinodchandran, and Osamu Watanabe. Superpolynomial versus half-exponential circuit size in the exponential hierarchy. In Takao Asano, Hiroshi Imai, D. T. Lee, Shin-Ichi Nakano, and Takeshi Tokuyama, editors, Computing and Combinatorics, 5th Annual International Conference, COCOON '99, Tokyo, Japan, July 26-28, 1999, Proceedings, volume 1627 of Lecture Notes in Computer Science, pages 210–220. Springer, 1999. doi:10.1007/3-540-48686-0_21.

[36] Ashish V. Naik, Kenneth W. Regan, and D. Sivakumar. On quasilinear-time complexity theory. Theor. Comput. Sci., 148(2):325–349, 1995. doi:10.1016/0304-3975(95)00031-Q.

[37] Noam Nisan and Avi Wigderson. Hardness vs randomness. J. Comput. Syst. Sci., 49(2):149–167, 1994. doi:10.1016/S0022-0000(05)80043-1.

[38] Igor C. Oliveira. Randomness and intractability in kolmogorov complexity. In Christel Baier, Ioannis Chatzigiannakis, Paola Flocchini, and Stefano Leonardi, editors, 46th International Colloquium on Automata, Languages, and Programming, ICALP 2019, July 9-12, 2019, Patras, Greece, volume 132 of LIPIcs, pages 32:1–32:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.ICALP.2019.32.

[39] Igor C. Oliveira and Rahul Santhanam. Pseudodeterministic constructions in subexponential time. In Hamed Hatami, Pierre McKenzie, and Valerie King, editors, Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, Montreal, QC, Canada, June 19-23, 2017, pages 665–677. ACM, 2017. doi:10.1145/3055399.3055500.

[40] Hanlin Ren and Rahul Santhanam. Hardness of KT characterizes parallel cryptography. In 36th Computational Complexity Conference, CCC 2021, July 20-23, 2021, Toronto, Ontario, Canada (Virtual Conference), pages 35:1–35:58, 2021. doi:10.4230/LIPIcs.CCC.2021.35.

[41] Shai Shalev-Shwartz and Shai Ben-David. Understanding Machine Learning - From Theory to Algorithms. Cambridge University Press, 2014. URL: http://www.cambridge.org/de/academic/subjects/computer-science/pattern-recognition-and-machine-learning/ understanding-machine-learning-theory-algorithms.

[42] Ronen Shaltiel and Christopher Umans. Simple extractors for all min-entropies and a new pseudorandom generator. J. ACM, 52(2):172–216, 2005. doi:10.1145/1059513.1059516.

[43] Luca Trevisan and Salil P. Vadhan. Pseudorandomness and average-case complexity via uniform reductions. Computational Complexity, 16(4):331–364, 2007.

[44] Salil P. Vadhan. On learning vs. refutation. In Satyen Kale and Ohad Shamir, editors, Proceed- ings of the 30th Conference on Learning Theory, COLT 2017, Amsterdam, The Netherlands, 7-10 July 2017, volume 65 of Proceedings of Machine Learning Research, pages 1835–1848. PMLR, 2017. URL: http://proceedings.mlr.press/v65/vadhan17a.html.

[45] Emanuele Viola. On constructing parallel pseudorandom generators from one-way functions. In 20th Annual IEEE Conference on Computational Complexity (CCC 2005), 11-15 June 2005, San Jose, CA, USA, pages 183–197, 2005. doi:10.1109/CCC.2005.16.